

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

«Методы оптимизации»

Домашнее задание
Вариант 1

Выполнили:

Козаченко Данил

Яруков Артём

Группа Р3212

Проверила:

Селина Елена Георгиевна

Санкт-Петербург 2026г.

Содержание

1	Часть 1	2
1.1	Метод кубической аппроксимации	2
1.2	Исходные данные	3
1.3	Код решения	3
1.4	Результаты работы алгоритма	4
2	Часть 2	7
2.1	Методы многомерной оптимизации	7
2.1.1	Метод покоординатного спуска	7
2.1.2	Метод градиентного спуска (с постоянным шагом)	7
2.1.3	Метод наискорейшего спуска	8
2.1.4	Метод Ньютона	8
2.2	ДЗ 2.1. Визуализации линий уровня и ломаной последовательных приближений	9
2.3	ДЗ 2.2. Метод Ньютона	9
2.4	Результаты работы алгоритма	11
2.5	ДЗ 2.3. Объектная модель задачи минимизации функции	11
3	Часть 3	16
3.1	Описание датасета	16
3.2	Метод оценки важности признаков на основе градиентного бустинга	16
3.3	Метод отбора признаков на основе муравьиного алгоритма (UFSACO)	18
3.4	Код решения	19
3.5	Вывод	26

1 Часть 1

1.1 Метод кубической аппроксимации

Метод кубической аппроксимации применяется для непрерывно дифференцируемых функций. Если на границах интервала $[x_1, x_2]$ выполняется условие:

$$y_{11} \cdot y_{12} < 0,$$

где $y_{11} = f'(x_1)$, $y_{12} = f'(x_2)$, то функция строго унимодальна на этом отрезке, и внутри него гарантированно существует точка минимума.

Для аппроксимации строится кубический интерполяционный многочлен Эрмита:

$$P_3(x) = a_0 + a_1(x - x_1) + a_2(x - x_1)(x - x_2) + a_3(x - x_1)^2(x - x_2).$$

Обозначим $y_1 = f(x_1)$, $y_2 = f(x_2)$, а также значения производных $y_{11} = f'(x_1)$, $y_{12} = f'(x_2)$. Коэффициенты многочлена определяются соотношениями:

$$\begin{aligned} a_0 &= y_1, & a_1 &= y_{11}, \\ a_2 &= \frac{y_2 - y_1}{(x_2 - x_1)^2} - \frac{y_{11}}{x_2 - x_1}, \\ a_3 &= \frac{y_{12} - y_{11}}{(x_2 - x_1)^2} - 2 \frac{y_2 - y_1}{(x_2 - x_1)^3}. \end{aligned}$$

Из необходимого условия экстремума многочлена $P'_3(x) = 0$ получается следующее квадратное уравнение относительно $(x - x_1)$:

$$3a_3(x - x_1)^2 - 2 \frac{y_{11} + y_{12} - 3a_1}{x_2 - x_1}(x - x_1) + y_{11} = 0.$$

Решением этого уравнения на интервале $[x_1, x_2]$ является точка x_0 :

$$x_0 = x_1 + \mu(x_2 - x_1),$$

где вспомогательный коэффициент $\mu \in (0, 1)$ вычисляется через параметры:

$$\begin{aligned} z &= y_{11} + y_{12} - 3 \frac{y_2 - y_1}{x_2 - x_1}, \\ \omega &= \sqrt{z^2 - y_{11}y_{12}}, \\ \mu &= \frac{\omega + z - y_{11}}{2\omega - y_{11} + y_{12}}. \end{aligned}$$

Обновление интервала (сужение): Если $f'(x_0) \neq 0$, то выбирается новый, более узкий отрезок:

- если $f'(x_0) < 0$, то новым интервалом становится $[x_0, x_2]$;
- если $f'(x_0) > 0$, то новым интервалом становится $[x_1, x_0]$.

Критерий останова: расчет завершается, когда модуль производной в найденной точке становится меньше заданной точности:

$$|f'(x_0)| \leq \varepsilon.$$

1.2 Исходные данные

Целевая функция:

$$f(x) = x^2 - 3x + x \ln x, \quad [a, b] = [1, 2].$$

Производная для метода кубической аппроксимации:

$$f'(x) = 2x - 2 + \ln x.$$

1.3 Код решения

```
1 def cubic_approximation(x1, x2, eps=0.0001, max_iter=10):
2     print(f"Исходный отрезок по варианту: [{x1}, {x2}]")
3
4
5     step = 0.2
6     while df(x1) * df(x2) >= 0:
7         # Расширяем отрезок в обе стороны.
8
9         x1 = max(0.001, x1 - step)
10        x2 = x2 + step
11        print(f" -> Производные одного знака. Расширяем границы: [{x1:.4f}, {x2:.4f}]")
12
13    print(f"ШАГ 0 выполнен! Точки найдены: x1={x1:.4f} (f'={df(x1):.4f}), x2={x2:.4f} (f'={df(x2):.4f})")
14
15    history = []
16    print(f"{'Итер.':<7} | {'Отрезок':<16} | {'x0 (x_new)':<10} | {'f(x0)':<10}")
17    print("-" * 60)
18
19    for i in range(1, max_iter + 1):
20
21        y1, y2 = func(x1), func(x2)
22        y11, y12 = df(x1), df(x2)
23
24        z = y11 + y12 - 3 * (y2 - y1) / (x2 - x1)
25        w = np.sqrt(z ** 2 - y11 * y12)
26        mu = (w + z - y11) / (2 * w - y11 + y12)
27
28
29        # Найти x0 по формуле (10). Вычислить f'(x0)
30
31        x_new = x1 + mu * (x2 - x1)
32
33        print(f"{'i':<7} | [{x1:.4f}, {x2:.4f}] | {x_new:<10.4f} | {func(x_new):<10.5f}")
34        history.append(x_new)
35
36    # График для ДЗ 1.2
37    if i <= 2:
38        cubic_poly = CubicHermiteSpline([x1, x2], [y1, y2], [y11, y12])
39        plot_approximation(cubic_poly, [x1, x2], [y1, y2], "Кубическая аппроксимация", i)
40
```

```

41     # Критерий останова если  $|f'(x_0)| \leq \text{eps}$ , конец
42     if abs(df(x_new)) <= eps:
43         print(f"\nМинимум найден:  $x^* = \{x\_new:.6f\}$ ,  $f(x^*) = \{func(x\_new):.6f\}$ ")
44         break
45
46
47     #Обновление границ отрезка (сужение)
48
49     if df(x_new) < 0:
50         x1 = x_new
51     else:
52         x2 = x_new
53
54     return history

```

1.4 Результаты работы алгоритма

===== КВАДРАТИЧНАЯ АППРОКСИМАЦИЯ (ЛРЗ) =====

Итер.	x_1	x_2	x_3	x_{bar}	$f(x_{\text{bar}})$	Погр. f	Погр. x
1	1.0000	1.5000	0.5000	1.0148	-1.99967	0.000165	0.014629
2	0.5000	1.0000	1.0148	1.0005	-2.00000	0.000000	0.000520
3	0.5000	1.0000	1.0005	1.0000	-2.00000	0.000000	0.000018

Минимум найден в точке $x = 1.000018$, $f(x) = -2.000000$, Итераций: 3

===== КУБИЧЕСКАЯ АППРОКСИМАЦИЯ (ДЗ 1.1) =====

Исходный отрезок: $[1.0, 2.0]$ -> расширен до $[0.8000, 2.2000]$

Итер.	Отрезок	x_0 (x_{new})	$f(x_0)$
1	$[0.8000, 2.2000]$	1.0057	-1.99995
2	$[0.8000, 1.0057]$	1.0000	-2.00000

Минимум найден: $x^* = 0.999984$, $f(x^*) = -2.000000$

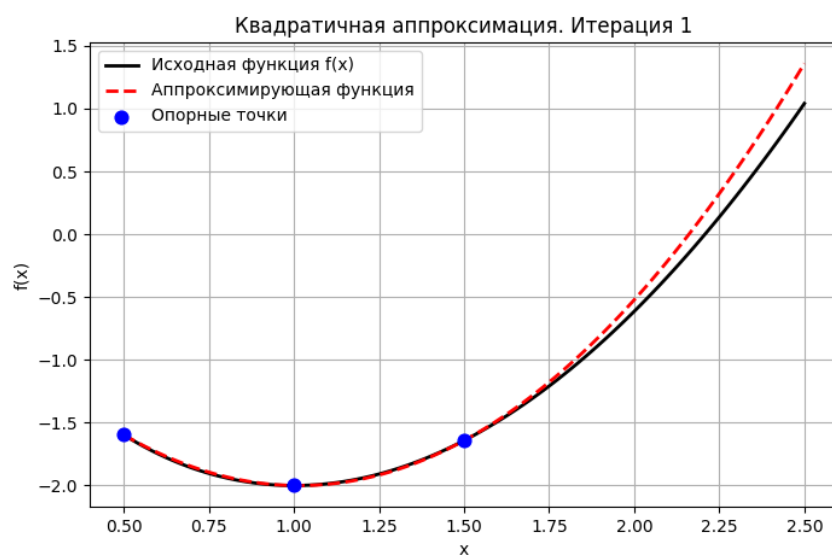


Рис. 1: Квадратичная аппроксимация

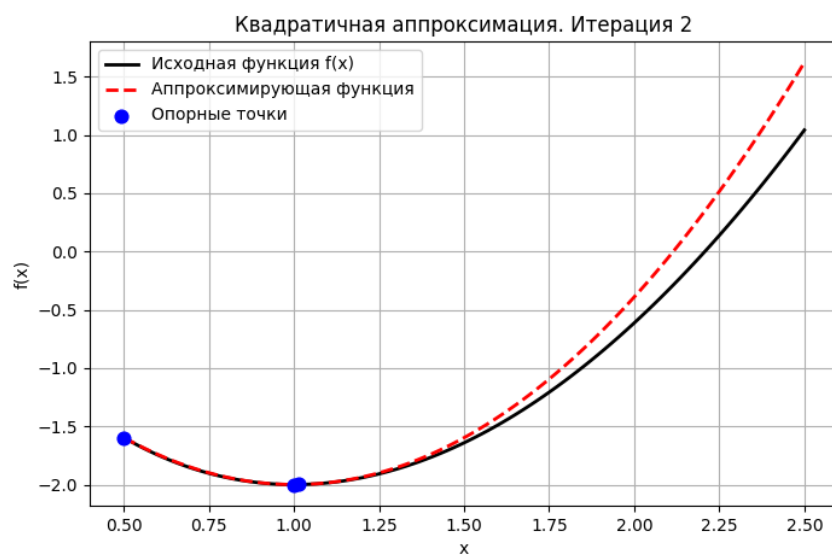


Рис. 2: Квадратичная аппроксимация

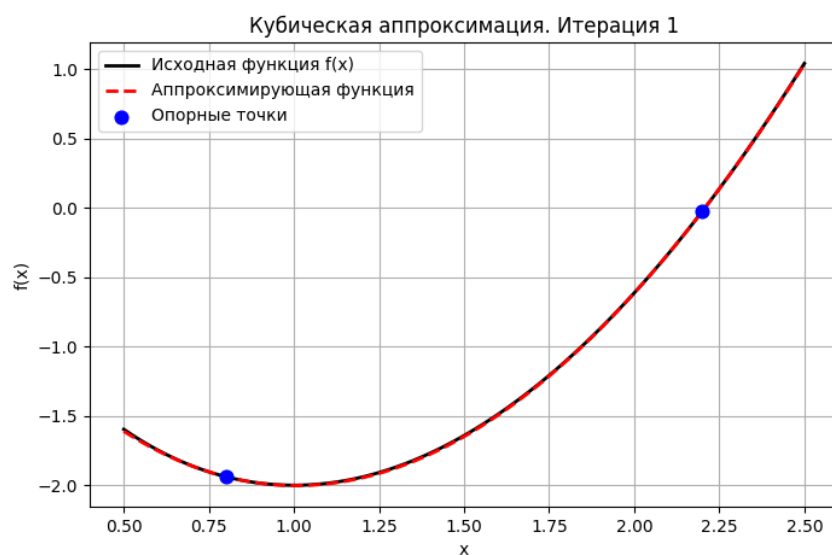


Рис. 3: Кубическая аппроксимация

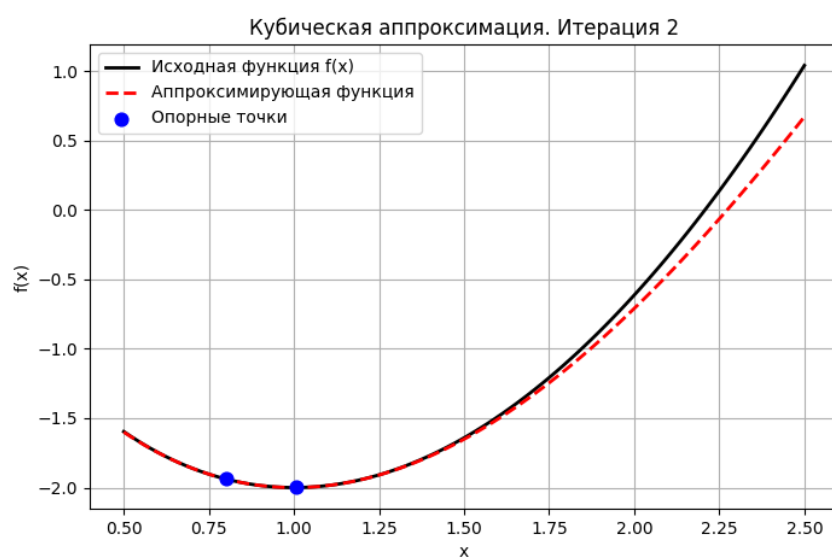


Рис. 4: Кубическая аппроксимация

2 Часть 2

2.1 Методы многомерной оптимизации

В задачах безусловной многомерной минимизации требуется найти точку $x^* \in \mathbb{R}^n$, в которой дифференцируемая целевая функция нескольких переменных $f(x) = f(x_1, x_2, \dots, x_n)$ принимает наименьшее значение:

$$f(x^*) = \min_{x \in \mathbb{R}^n} f(x).$$

2.1.1 Метод покоординатного спуска

Метод относится к методам прямого поиска (нулевого порядка), использующим только информацию о значениях целевой функции. Идея метода заключается в сведении многомерной задачи оптимизации к последовательному решению серии одномерных задач путем поочередного изменения координат.

Минимизация осуществляется циклами, состоящими из n шагов каждый. Пусть на итерации k задана точка $x^k = (x_1^k, x_2^k, \dots, x_n^k)^T$.

- **Шаг 1.** Фиксируются переменные $x_2^k, \dots, x_n^k$. Проводится минимизация по переменной x_1 :

$$x_1^{k+1} = \arg \min_{x_1} f(x_1, x_2^k, \dots, x_n^k).$$

Получаем точку $(x_1^{k+1}, x_2^k, \dots, x_n^k)^T$.

- **Шаг 2.** Фиксируются переменные $x_1^{k+1}, x_3^k, \dots, x_n^k$. Проводится минимизация по переменной x_2 :

$$x_2^{k+1} = \arg \min_{x_2} f(x_1^{k+1}, x_2, x_3^k, \dots, x_n^k).$$

Получаем точку $(x_1^{k+1}, x_2^{k+1}, x_3^k, \dots, x_n^k)^T$.

- **Шаг n .** Фиксируются переменные $x_1^{k+1}, \dots, x_{n-1}^{k+1}$. Проводится минимизация по последней переменной x_n :

$$x_n^{k+1} = \arg \min_{x_n} f(x_1^{k+1}, \dots, x_{n-1}^{k+1}, x_n).$$

Получаем точку $x^{k+1} = (x_1^{k+1}, x_2^{k+1}, \dots, x_n^{k+1})^T$, завершая цикл k .

Критерий останова: расчет прекращается при выполнении условий малости изменения функции или координат на двух соседних циклах:

$$|f(x^{k+1}) - f(x^k)| \leq \varepsilon \quad \text{или} \quad \|x^{k+1} - x^k\| \leq \varepsilon.$$

2.1.2 Метод градиентного спуска (с постоянным шагом)

Метод использует информацию первого порядка — градиент функции $\nabla f(x)$, который указывает направление наискорейшего возрастания функции:

$$\nabla f(x) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)^T.$$

Направление, противоположное градиенту (антиградиент) $-\nabla f(x)$, определяет направление наискорейшего убывания функции. Формула итерационного перехода имеет вид:

$$x^{(k+1)} = x^{(k)} - \lambda \nabla f(x^{(k)}),$$

где $\lambda > 0$ — постоянный шаг итерации (в координатной форме для двумерного случая: $x_i^{(k+1)} = x_i^{(k)} - \lambda \frac{\partial f}{\partial x_i}(x^{(k)})$).

Критерий останова: поиск завершается, когда норма градиента становится меньше заданной точности:

$$\|\nabla f(x^{(k)})\| \leq \varepsilon.$$

2.1.3 Метод наискорейшего спуска

В отличие от градиентного спуска с постоянным шагом, в методе наискорейшего спуска величина шага λ_k на каждой итерации выбирается оптимальной. Для этого решается задача одномерной минимизации целевой функции вдоль направления антиградиента:

$$S^{(k)} = -\nabla f(x^{(k)}).$$

Если направление нормируется, то:

$$S^{(k)} = -\frac{\nabla f(x^{(k)})}{\|\nabla f(x^{(k)})\|}.$$

Оптимальный шаг λ_k находится из условия:

$$\lambda_k = \arg \min_{\lambda > 0} f(x^{(k)} + \lambda S^{(k)}).$$

Новая точка рассчитывается как:

$$x^{(k+1)} = x^{(k)} + \lambda_k S^{(k)}.$$

Критерий останова: критерием окончания поиска служит малость модуля градиента:

$$\|\nabla f(x^{(k)})\| \leq \varepsilon.$$

2.1.4 Метод Ньютона

Метод Ньютона является методом второго порядка. Он основан на идее замены минимизируемой функции $f(x)$ в окрестности текущей точки $x^{(k)}$ квадратичной частью $\tilde{f}_k(x)$ ее разложения в ряд Тейлора:

$$\tilde{f}_k(x) = f(x^{(k)}) + \langle f'(x^{(k)}), (x - x^{(k)}) \rangle + \frac{1}{2} \langle (x - x^{(k)}) f''(x^{(k)}), (x - x^{(k)}) \rangle.$$

Для определения направления и величины шага используется матрица вторых частных производных (матрица Гессе) $f''(x^{(k)})$:

$$f''(x) = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{pmatrix}.$$

В методе Ньютона очередная точка $x^{(k+1)}$ в последовательности приближений к точке минимума выбирается по правилу:

$$x^{(k+1)} = x^{(k)} + \Delta x^{(k)} = x^{(k)} - [f''(x^{(k)})]^{-1} f'(x^{(k)}),$$

где $[f''(x^{(k)})]^{-1}$ — матрица, обратная к матрице Гессе, а $f'(x^{(k)})$ — вектор-градиент функции в точке $x^{(k)}$.

Критерий останова: расчет прекращается при выполнении условия:

$$\|\nabla f(x^{(k)})\| \leq \varepsilon.$$

2.2 ДЗ 2.1. Визуализации линий уровня и ломаной последовательных приближений

Целевая функция:

$$f(x_1, x_2) = 2x_1^2 + 2x_1x_2 + 3x_2^2 - 10x_1 - 10x_2 + 15$$

Начальное приближение $(-2; -2)$

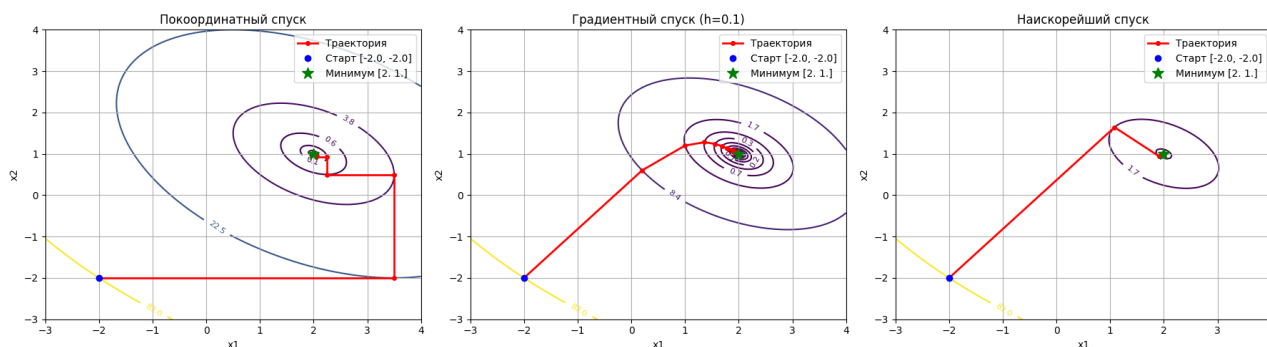


Рис. 5: Результаты работы алгоритмов

2.3 ДЗ 2.2. Метод Ньютона

Найдём экстремумы функции аналитически

Целевая функция:

$$z(x, y) = x^3 + 3x^2 + y^3 - 14y^2 + 64y - 100, \quad M_0 = (-0.5, 5.5).$$

Найдём стационарные точки

$$z_x = \frac{\partial z}{\partial x} = 3x^2 + 6x,$$

$$z_y = \frac{\partial z}{\partial y} = 3y^2 - 28y + 64.$$

Приравнивая производные к нулю, получаем:

$$\begin{cases} 3x^2 + 6x = 0, \\ 3y^2 - 28y + 64 = 0. \end{cases}$$

Решаем первое уравнение системы:

$$3x(x + 2) = 0 \implies x_1 = 0, \quad x_2 = -2.$$

Решаем второе уравнение системы:

$$3y^2 - 28y + 64 = 0 \implies y_1 = 4, \quad y_2 = \frac{16}{3} \approx 5.33$$

Стационарные точки:

$$M_1(0, 4), \quad M_2\left(0, \frac{16}{3}\right), \quad M_3(-2, 4), \quad M_4\left(-2, \frac{16}{3}\right).$$

Проверим достаточное условие экстремума:

$$A = z_{xx} = \frac{\partial^2 z}{\partial x^2} = 6x + 6,$$

$$B = z_{xy} = z_{yx} = \frac{\partial^2 z}{\partial x \partial y} = 0,$$

$$C = z_{yy} = \frac{\partial^2 z}{\partial y^2} = 6y - 28.$$

$$H(x, y) = \begin{pmatrix} z_{xx} & z_{xy} \\ z_{yx} & z_{yy} \end{pmatrix} = \begin{pmatrix} 6x + 6 & 0 \\ 0 & 6y - 28 \end{pmatrix}.$$

Определитель матрицы Гессе равен:

$$\Delta = \det H(x, y) = A \cdot C - B^2 = (6x + 6)(6y - 28).$$

Исследуем каждую из четырех точек:

- Точка $M_1(0, 4)$:

$$A = 6(0) + 6 = 6 > 0, \quad C = 6(4) - 28 = -4,$$

$$\Delta = 6 \cdot (-4) = -24 < 0.$$

Так как определитель $\Delta < 0$, точка $M_1(0, 4)$ является **седловой точкой**.

- Точка $M_2\left(0, \frac{16}{3}\right)$:

$$A = 6(0) + 6 = 6 > 0, \quad C = 6 \cdot \frac{16}{3} - 28 = 32 - 28 = 4 > 0,$$

$$\Delta = 6 \cdot 4 = 24 > 0.$$

Так как определитель $\Delta > 0$ и $A > 0$, точка $M_2\left(0, \frac{16}{3}\right)$ является **точкой локального минимума**.

- Точка $M_3(-2, 4)$:

$$A = 6(-2) + 6 = -6 < 0, \quad C = 6(4) - 28 = -4 < 0,$$

$$\Delta = (-6) \cdot (-4) = 24 > 0.$$

Так как определитель $\Delta > 0$ и $A < 0$, точка $M_3(-2, 4)$ является **точкой локального максимума**.

- Точка $M_4\left(-2, \frac{16}{3}\right)$:

$$A = 6(-2) + 6 = -6 < 0, \quad C = 6 \cdot \frac{16}{3} - 28 = 32 - 28 = 4 > 0,$$

$$\Delta = (-6) \cdot 4 = -24 < 0.$$

Так как определитель $\Delta < 0$, точка $M_4\left(-2, \frac{16}{3}\right)$ является **седловой точкой**.

Программное решение

```

def newton_method(x0, eps=0.0001, max_iter=50):
    print("Метод Ньютона")
    x = np.array(x0, dtype=float)
    g = grad_z(x)
    norm_g = np.linalg.norm(g)
    f_val = z_func(x)
    print(
        f"{'Итер.':<5} | {'dx1':<8} | {'dx2':<8} | {'x':<8} | {'y':<8} | {'f(x,y)':<10}"
    )
    print("-" * 95)
    print(
        f"{'0':<5} | {'-':<8} | {'-':<8} | {x[0]:<8.4f} | {x[1]:<8.4f} | {f_val:<10.4f} |"
    )
    for k in range(1, max_iter + 1):
        H = hessian_z(x)
        H_inv = np.linalg.inv(H)

        delta_x = -np.dot(H_inv, g)
        x = x + delta_x

        g = grad_z(x)
        norm_g = np.linalg.norm(g)
        f_val = z_func(x)

        print(
            f"{'k':<5} | {delta_x[0]:<8.4f} | {delta_x[1]:<8.4f} | {x[0]:<8.4f} | {x[1]:<8.4f} |"
        )
        if norm_g < eps:
            break
    return x

```

2.4 Результаты работы алгоритма

Итер.	dx1	dx2	x	y	f(x,y)	dz/dx	dz/dy	
0	-	-	-0.5000	5.5000	-4.5000	-2.2500	0.7500	2
1	0.7500	-0.1500	0.2500	5.3500	-4.9815	1.6875	0.0675	1
2	-0.2250	-0.0165	0.0250	5.3335	-5.1833	0.1519	0.0008	0
3	-0.0247	-0.0002	0.0003	5.3333	-5.1852	0.0018	0.0000	0
4	-0.0003	-0.0000	0.0000	5.3333	-5.1852	0.0000	0.0000	0

Итоговая точка минимума: x = 0.000000, y = 5.333333

2.5 ДЗ 2.3. Объектная модель задачи минимизации функции

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.optimize import minimize_scalar
4 from abc import ABC, abstractmethod
5
6
7 class OptimizationProblem:

```

```

8     """Класс, инкапсулирующий целевую функцию и её градиент"""
9
10    def __init__(self, func, grad):
11        self.func = func
12        self.grad = grad
13
14
15    class Optimizer(ABC):
16        """Абстрактный базовый класс для методов оптимизации"""
17
18        def __init__(self, eps=0.0001, max_iter=50):
19            self.eps = eps
20            self.max_iter = max_iter
21            self.history = []
22
23        @abstractmethod
24        def optimize(self, problem: OptimizationProblem, x0):
25            pass
26
27
28    class CoordinateDescent(Optimizer):
29        """Класс метода покоординатного спуска"""
30
31        def optimize(self, problem: OptimizationProblem, x0):
32            x = np.array(x0, dtype=float)
33            self.history = [x.copy()]
34
35            for _ in range(self.max_iter):
36                x_prev = x.copy()
37
38                # Шаг по x1
39                res1 = minimize_scalar(lambda alpha: problem.func([x[0] + alpha, x[1]]))
40                x[0] += res1.x
41                self.history.append(x.copy())
42
43                # Шаг по x2
44                res2 = minimize_scalar(lambda alpha: problem.func([x[0], x[1] + alpha]))
45                x[1] += res2.x
46                self.history.append(x.copy())
47
48                if np.linalg.norm(x - x_prev) < self.eps:
49                    break
50            return x
51
52
53    class GradientDescent(Optimizer):
54        """Класс метода градиентного спуска"""
55
56        def __init__(self, learning_rate=0.1, eps=0.0001, max_iter=50):
57            super().__init__(eps, max_iter)
58            self.learning_rate = learning_rate

```

```

59
60 def optimize(self, problem: OptimizationProblem, x0):
61     x = np.array(x0, dtype=float)
62     self.history = [x.copy()]
63
64     for _ in range(self.max_iter):
65         g = problem.grad(x)
66         if np.linalg.norm(g) < self.eps:
67             break
68         x = x - self.learning_rate * g
69         self.history.append(x.copy())
70     return x
71
72
73 class SteepestDescent(Optimizer):
74     """Класс метода наискорейшего спуска"""
75
76     def optimize(self, problem: OptimizationProblem, x0):
77         x = np.array(x0, dtype=float)
78         self.history = [x.copy()]
79
80         for _ in range(self.max_iter):
81             g = problem.grad(x)
82             if np.linalg.norm(g) < self.eps:
83                 break
84
85             res = minimize_scalar(lambda h: problem.func(x - h * g))
86             x = x - res.x * g
87             self.history.append(x.copy())
88         return x
89
90
91 def f_var1(X):
92     return 2 * X[0] ** 2 + 2 * X[0] * X[1] + 3 * X[1] ** 2 - 10 * X[0] - 10 * X[1] + 15
93
94
95 def grad_f_var1(X):
96     return np.array([4 * X[0] + 2 * X[1] - 10, 2 * X[0] + 6 * X[1] - 10])
97
98
99 def z_func(X):
100     x, y = X
101     return x ** 3 + 3 * x ** 2 + y ** 3 - 14 * y ** 2 + 64 * y - 100
102
103
104 def grad_z(X):
105     x, y = X
106     return np.array([3 * x ** 2 + 6 * x, 3 * y ** 2 - 28 * y + 64])
107
108
109 def hessian_z(X):

```

```

110     x, y = X
111     return np.array([[6 * x + 6, 0], [0, 6 * y - 28]])
112
113
114 def newton_method(x0, eps=0.0001, max_iter=50):
115     print("Метод Ньютона")
116     x = np.array(x0, dtype=float)
117     g = grad_z(x)
118     norm_g = np.linalg.norm(g)
119     f_val = z_func(x)
120     print(
121         f"{'Итер.':<5} | {'dx1':<8} | {'dx2':<8} | {'x':<8} | {'y':<8} | {'f(x,y)':<10} | {'dz/dx':<8}"
122     )
123     print("-" * 95)
124     print(
125         f"{'0':<5} | {'-':<8} | {'-':<8} | {x[0]:<8.4f} | {x[1]:<8.4f} | {f_val:<10.4f} | {g[0]:<8.4f} |"
126     )
127     for k in range(1, max_iter + 1):
128         H = hessian_z(x)
129         H_inv = np.linalg.inv(H)
130
131         delta_x = -np.dot(H_inv, g)
132         x = x + delta_x
133
134         g = grad_z(x)
135         norm_g = np.linalg.norm(g)
136         f_val = z_func(x)
137
138         print(
139             f"{'k':<5} | {delta_x[0]:<8.4f} | {delta_x[1]:<8.4f} | {x[0]:<8.4f} | {x[1]:<8.4f} | {f_val:<10.4f} |"
140         )
141         if norm_g < eps:
142             break
143     return x
144
145 def main():
146     problem = OptimizationProblem(f_var1, grad_f_var1)
147     x_start = [-2.0, -2.0]
148
149     optimizers = {
150         "Покоординатный спуск": CoordinateDescent(),
151         "Градиентный спуск (h=0.1)": GradientDescent(learning_rate=0.1),
152         "Наискорейший спуск": SteepestDescent()
153     }
154
155     fig, axes = plt.subplots(1, 3, figsize=(18, 5))
156     X1, X2 = np.meshgrid(np.linspace(-3, 4, 150), np.linspace(-3, 4, 150))
157     Z = f_var1([X1, X2])
158
159     for ax, (title, optimizer) in zip(axes, optimizers.items()):
160         optimizer.optimize(problem, x_start)
161         hist = np.array(optimizer.history)

```

```

161     # Считаем значения функции в каждой точке траектории
162     func_values = [problem.func(pt) for pt in hist]
163     unique_levels = sorted(list(set(np.round(func_values, 4))))
164     # Отрисовка линий уровня
165     contours = ax.contour(X1, X2, Z, levels=unique_levels, cmap='viridis', zorder=1)
166     ax.clabel(contours, inline=True, fontsize=8, fmt='%.1f')
167     # Отрисовка траектории
168     ax.plot(hist[:, 0], hist[:, 1], 'r.-', linewidth=2, markersize=8, label='Траектория', zorder=2)
169     ax.plot(hist[0, 0], hist[0, 1], 'bo', label=f'Старт {x_start}', zorder=3)
170     ax.plot(hist[-1, 0], hist[-1, 1], 'g*', markersize=12, label=f'Минимум {np.round(hist[-1, 2])}')
171
172     ax.set_title(title)
173     ax.set_xlabel("x1")
174     ax.set_ylabel("x2")
175     ax.legend()
176     ax.grid(True)
177
178     plt.tight_layout()
179     plt.show()
180
181     M0 = [-0.5, 5.5]
182     optimum = newton_method(M0)
183     print(f"Итоговая точка минимума: x = {optimum[0]:.6f}, y = {optimum[1]:.6f}")
184
185
186 if __name__ == '__main__':
187     main()
188

```

3 Часть 3

3.1 Описание датасета

В качестве исследуемого набора данных выбран датасет [Strategic Voting Machine](#)

Предметная область: Математическое моделирование результатов голосования. Данный датасет представляет собой многомерный массив данных, содержащий информацию о ходе выборов на различных участках.

Постановка задачи регрессии: В качестве целевой переменной выбран вещественный признак `Vote_Percentage` — итоговый процент голосов, полученный кандидатом на избирательном участке. Признак `Voter_Turnout` (явка) был исключен из рассмотрения на этапе предобработки (`CONNECTED_COLS`), так как он имеет прямую математическую связь с целевым признаком).

- **Размерность данных:** Исходный датасет содержит 22 000 объектов и 62 признака. После удаления объектов с пропущенными значениями (NaN) выборка сократилась до 13 285 объектов
- **Пространство признаков:** После фильтрации нечисловых столбцов и удаления целевой переменной было отобрано 44 числовых признака. Их можно разделить на следующие ключевые группы:
 1. *Масштабные характеристики участка:* `Registered_Voters` (число зарегистрированных избирателей), `Total_Votes` (общее число проголосовавших).
 2. *Каналы голосования:* `EVM_Votes` (электронное голосование), `Postal_Votes` (почтовые бюллетени).
 3. *Финансовые и медийные показатели:* `Campaign_Spending_Cr` (расходы на кампанию), `Social_Media_Followers` (подписчики кандидата в соцсетях), `Fake_News_Score` (индекс распространения дезинформации).
 4. *Индикаторы рисков и аномалий:* `Vote_Surge_Score` (индекс резкого всплеска голосов), `Booth_Capture_Risk` (риск захвата избирательного участка), `Fraud_Proxy_Duplicate` (выявленные дубликаты голосов).

3.2 Метод оценки важности признаков на основе градиентного бустинга

Итоговое предсказание алгоритма градиентного бустинга формируется как сумма предсказаний последовательно построенных деревьев решений:

$$F(X) = \sum_{m=1}^M f_m(X),$$

где используются следующие переменные:

- X — вектор входных признаков объекта (в нашей задаче — характеристики конкретного избирательного участка);
- $F(X)$ — итоговое предсказание модели (прогнозируемый процент голосов `Vote_Percentage`);
- M — общее количество деревьев в ансамбле (в нашем коде задано параметром `n_estimators = 300`);

- m — порядковый номер текущего дерева ($m = 1, 2, \dots, M$);
- $f_m(X)$ — предсказание отдельного (базового) дерева с номером m , которое обучается предсказывать и исправлять ошибки, допущенные суммой всех предыдущих деревьев.

Оценка важности каждого отдельного признака вычисляется непосредственно в процессе построения этих деревьев. Когда алгоритм строит дерево, он разбивает данные в каждом узле на две ветви (левую и правую). Снижение ошибки при разбиении родительского узла t по некоторому признаку j вычисляется по формуле:

$$\Delta I(t) = N_t \cdot D(t) - N_l \cdot D(l) - N_r \cdot D(r),$$

где:

- t — родительский узел дерева до разбиения;
- l и r — левый и правый дочерние узлы, образовавшиеся после разбиения узла t ;
- $\Delta I(t)$ — абсолютное снижение ошибки (неопределенности) в результате этого разбиения;
- N_t, N_l, N_r — количество объектов (избирательных участков), попавших в узлы t, l и r соответственно ($N_t = N_l + N_r$);
- $D(t), D(l), D(r)$ — дисперсия (мера ошибки) целевой переменной в соответствующих узлах.

Сама дисперсия $D(t)$ для узла t в задаче регрессии рассчитывается как среднее квадратичное отклонение:

$$D(t) = \frac{1}{N_t} \sum_{i \in t} (y_i - \bar{y}_t)^2, \quad \bar{y}_t = \frac{1}{N_t} \sum_{i \in t} y_i,$$

где:

- i — индекс конкретного объекта (участка), попавшего в узел t ;
- y_i — истинное значение целевой переменной (`Vote_Percentage`) для объекта i ;
- $\bar{y}_t$ — среднее значение целевой переменной для всех объектов внутри узла t .

Суммарная важность конкретного признака j внутри **одного дерева** T вычисляется как сумма всех снижений ошибок во всех внутренних узлах этого дерева, где данные были разделены именно по признаку j :

$$\text{Imp}_j(T) = \sum_{t \in T \text{ (если разбиение по признаку } j)} \Delta I(t).$$

Финальное значение важности признака j во **всем ансамбле** представляет собой среднюю важность этого признака по всем M деревьям:

$$\text{Imp}_j = \frac{1}{M} \sum_{m=1}^M \text{Imp}_j(f_m).$$

Полученные значения Imp_j масштабируются (нормируются) так, чтобы их сумма по всем 44 признакам датасета равнялась 1 (или 100%). Таким образом, чем чаще признак используется для разбиения и чем сильнее он снижает ошибку (дисперсию) модели, тем выше его итоговая важность на графике.

3.3 Метод отбора признаков на основе муравьиного алгоритма (UFSACO)

В неконтролируемом методе отбора признаков UFSACO (Unsupervised Feature Selection based on Ant Colony Optimization) пространство поиска представляется в виде полносвязного ненаправленного графа $G = \langle F, E \rangle$. В этой графовой модели используются следующие обозначения:

- G — граф, представляющий пространство поиска;
- $F = \{F_1, F_2, \dots, F_n\}$ — множество узлов графа, где каждый узел F_i соответствует одному из n исходных признаков датасета (в нашем коде $n = 44$);
- E — множество ребер, соединяющих каждую пару признаков.

Вес каждого ребра E между признаками F_i и F_j выражает абсолютную величину их косинусной схожести (similarity), которая вычисляется по формуле:

$$\text{sim}(F_i, F_j) = \left| \frac{\sum_{m=1}^N f_{im} f_{jm}}{\sqrt{\sum_{m=1}^N f_{im}^2 \sum_{m=1}^N f_{jm}^2}} \right| \in [0, 1],$$

где:

- N — количество объектов (строк) в датасете;
- m — индекс конкретного объекта ($m = 1, 2, \dots, N$);
- f_{im}, f_{jm} — нормированные значения признаков F_i и F_j для объекта m .

Эвристическая привлекательность перехода $\eta(F_i, F_j)$ представляет собой меру **непохожести** признаков (чем меньше признаки похожи, тем больше информации они дадут вместе). Она вычисляется как величина, обратная косинусной схожести:

$$\eta(F_i, F_j) = \frac{1}{\text{sim}(F_i, F_j)}.$$

Этапы работы алгоритма:

1. **Инициализация:** На старте нулевой итерации ($t = 0$) каждому узлу-признаку i присваивается одинаковое стартовое количество феромона τ_i :

$$\tau_i(0) = \text{const} \in (0, 1).$$

В коде эта константа задана параметром `INIT_PHEROMONE`.

2. **Выбор пути агентами (переходы):** На каждой итерации K искусственных муравьев (в коде `N_ANTS`) случайным образом помещаются на стартовые узлы. Каждый k -й муравей ($k = 1, \dots, K$), находящийся на узле i , осуществляет переход к следующему узлу j на основе сравнения случайного числа $q \in [0, 1]$ с заданным порогом q_0 (в коде `EXPLOITATION_PROB`):

- **Режим применения опыта (exploitation), если $q \leq q_0$:** осуществляется жадный (детерминированный) переход на узел с максимальным значением комбинированной привлекательности:

$$j = \arg \max_{u \in J_k^i} (\tau_u \cdot [\eta(F_i, F_u)]^\alpha).$$

- **Режим исследования (exploration), если $q > q_0$:** переход носит стохастический характер. Вероятность перехода муравья из узла i в узел j равна:

$$P_k(i, j) = \begin{cases} \frac{\tau_j \cdot [\eta(F_i, F_j)]^\alpha}{\sum_{u \in J_k^i} \tau_u \cdot [\eta(F_i, F_u)]^\alpha}, & \text{если } j \in J_k^i, \\ 0, & \text{если } j \notin J_k^i. \end{cases}$$

Где используются следующие переменные:

- j — выбираемый целевой узел (признак);
- J_k^i — множество узлов, которые k -й муравей еще **не посетил** в рамках текущего цикла;
- u — индекс перебора по всем непосещенным узлам из J_k^i ;
- τ_u, τ_j — текущее количество феромона на узлах u и j ;
- α — гиперпараметр (в коде `ALPHA`), регулирующий степень влияния эвристической информации (непохожести) по отношению к уровню феромона.

3. **Обновление феромонов:** После того как все муравьи завершили свои обходы в рамках итерации (эпохи) t , количество феромона на каждом узле i пересчитывается. Сначала часть старого феромона испаряется, а затем добавляется новый след:

$$\tau_i(t+1) = (1 - \rho)\tau_i(t) + \Delta\tau_i(t).$$

Величина добавляемого феромона $\Delta\tau_i(t)$ определяется относительной частотой посещения данного узла муравьями:

$$\Delta\tau_i(t) = \frac{FC(i)}{\sum_{m=1}^n FC(m)}.$$

Переменные на этапе обновления:

- $\tau_i(t+1), \tau_i(t)$ — количество феромона на узле i на новой и старой итерациях соответственно;
- ρ — коэффициент испарения феромона (в коде `RO`), $\rho \in (0, 1)$;
- $FC(i)$ (*Feature Count*) — абсолютное количество раз, которое признак i был выбран всеми агентами за текущую итерацию;
- $\sum_{m=1}^n FC(m)$ — общее количество всех переходов, совершенных муравьями за итерацию (сумма посещений всех узлов).

3.4 Код решения

```

1  import copy
2  import numpy as np
3  import pandas as pd
4  import matplotlib.pyplot as plt
5  import seaborn as sns
6  from sklearn import ensemble
7  from sklearn.preprocessing import MinMaxScaler
8
9
10 def load_and_preprocess_data(filepath, target_col, connected_cols, max_objects=5000):
11     """
12     Читаем датасет и выбираем только числовые признаки, исключая TARGET

```

```

13 """
14 raw = pd.read_csv(filepath)
15 print("Размер исходного датасета:", raw.shape)
16
17 numeric_cols = raw.select_dtypes(include=np.number).columns.tolist()
18 features = [
19     col for col in numeric_cols
20     if col != target_col and col not in connected_cols
21 ]
22
23 df = raw[[target_col] + features].copy()
24 df = df.dropna().reset_index(drop=True)
25 print("Количество объектов после удаления пропусков:", len(df))
26 print("Количество числовых признаков без target:", len(features))
27 print("Размер данных для расчета:", df.shape)
28 print(df.describe())
29
30 # Ограничиваем выборку, тк иначе будет очень долго работать
31 if max_objects is not None and len(df) > max_objects:
32     df = df.sample(n=max_objects, random_state=42).reset_index(drop=True)
33     print(f"Выборка ограничена до {max_objects} объектов.")
34
35 return df, features
36
37
38 def scale_features(df, features, target_col):
39     """
40     Нормирование входных признаков методом MinMaxScaler
41     """
42     x_train = df[features].copy()
43     y_train = df[target_col].copy()
44
45     x_scaler = MinMaxScaler()
46     x_scaler.fit(x_train)
47     x_train_scaled = x_scaler.transform(x_train)
48
49     print("X shape:", x_train_scaled.shape)
50     print("y shape:", y_train.shape)
51
52     return x_train_scaled, y_train
53
54
55 def run_gradient_boosting(x_train, y_train, features, n_features=15):
56     """
57     Обучение модели GradientBoostingRegressor и отбор n_features наиболее важных признаков.
58     """
59     params = {
60         "n_estimators": 300,          # количество деревьев
61         "max_depth": 4,              # максимальная глубина каждого дерева
62         "min_samples_split": 10,     # минимальное количество объектов, нужное для разбиения узла дерева
63         "learning_rate": 0.01,       # скорость обучения
64         "verbose": 0,                # (конфигурационный параметр) без подробного вывода процесса обучения
65     }
66     model_base_full = ensemble.GradientBoostingRegressor(**params)
67     model_base_full.fit(x_train, y_train)
68
69     base_feature_importance = copy.deepcopy(model_base_full.feature_importances_)
70     base_feature_importance_sorted_idx = np.argsort(base_feature_importance)[::-1]
71     base_feature_importance_sorted_idx = base_feature_importance_sorted_idx[0:n_features]
72
73     boosting_names = [features[i] for i in base_feature_importance_sorted_idx]
74     vals = base_feature_importance[base_feature_importance_sorted_idx]
75     fimps = pd.DataFrame(data={"Name": boosting_names, "Vals": vals})
76
77     return boosting_names, fimps

```

```

78
79
80 def plot_boosting_importance(fimps):
81     """Визуализация важности признаков Gradient Boosting."""
82     sns.set_context("paper", font_scale=1.3)
83     plt.figure(figsize=(10, 7))
84     sns.barplot(x="Vals", y="Name", data=fimps, label="Total", color="b")
85     plt.title("GradientBoostingRegressor: feature_importances_")
86     plt.tight_layout()
87     plt.show()
88
89
90 def get_sim(all_scaled, i, j, sim_cache, eps=1e-6):
91     """
92     Возвращает косинусное сходство между признаками i и j,
93     используя кэширование в переданном словаре sim_cache.
94     """
95     pair = (min(i, j), max(i, j))
96     if pair not in sim_cache:
97         a = all_scaled[:, i]
98         b = all_scaled[:, j]
99         denom = (np.linalg.norm(a) * np.linalg.norm(b)) + eps
100        res = np.dot(np.asarray(a), np.asarray(b)) / denom
101        sim_cache[pair] = np.abs(res)
102    return sim_cache[pair]
103
104
105 def run_ufsaaco(all_scaled, params, sim_cache, eps=1e-6, verbose=False):
106     """
107     Реализация метода муравьиной колонии для отбора признаков (UFSACO).
108     """
109     n_start_features = all_scaled.shape[1]
110     nc_max = params["NC_MAX"]
111     n_steps = params["N_STEPS"]
112     init_pheromone = params["INIT_PHEROMONE"]
113     ro = params["RO"]
114     exploitation_prob = params["EXPLOITATION_PROB"]
115     alpha = params["ALPHA"]
116     n_ants = params["N_ANTS"]
117
118     # Иницизируем начальным значением феромона в каждом узле
119     tau = init_pheromone * np.ones((n_start_features))
120
121     # Внешний цикл - число эпох
122     for count in range(nc_max):
123         # Случайно размещаем муравьев по узлам с вероятностью, пропорциональной феромону
124         tau_sum = sum(tau)
125         p_dist = tau / tau_sum if tau_sum > 0 else np.ones_like(tau) / n_start_features
126         ants_pos = np.random.choice(n_start_features, size=n_ants, p=p_dist)
127
128         # Очищаем счетчик посещений узлов для каждого узла
129         visits = np.zeros((n_start_features))
130
131         # Очищаем множество посещенных узлов каждым k-ым агентом из каждого i-ого узла
132         nodes_visited = {(k, i): set() for k in range(n_ants) for i in range(n_start_features)}
133
134         # Внутренний цикл - длина пути, который проходят все муравьи
135         for iter in range(n_steps):
136             # k - номер текущего муравья
137             for k in range(n_ants):
138                 # На каком i-ом узле находится k-ый муравей
139                 i = ants_pos[k]
140
141                 # Множество посещенных узлов
142                 visited = nodes_visited[(k, i)]

```

```

143
144     # Множество непосещенных узлов
145     unvisited = list((set(range(n_start_features)) - visited) - {i})
146
147     if not unvisited:
148         continue
149
150     # Вычисляем желательность перехода к непосещенным узлам
151     node_score = np.array([
152         tau[j] / (np.power(get_sim(all_scaled, i, j, sim_cache, eps), alpha) + eps)
153         for j in unvisited
154     ])
155
156     # Какой шаг выполняем - exploration или exploitation?
157     q = np.random.uniform()
158     if q <= exploitation_prob:
159         # EXPLOITATION (жадный выбор самого привлекательного)
160         if verbose:
161             print("EXPLOITATION")
162         # Переходим в узел с максимальной желательностью
163         jj = np.argmax(node_score)
164     else:
165         # EXPLORATION (стохастический выбор пропорционально вероятностям)
166         if verbose:
167             print("EXPLORATION")
168         # Переходим по вероятности, пропорциональной желательности
169         total_score = node_score.sum()
170         if total_score <= 0:
171             # вырожденный случай: равномерное распределение
172             p = None
173         else:
174             p = node_score / total_score
175         jj = np.random.choice(len(unvisited), size=1, p=p)[0]
176
177     # Получаем номер следующего узла j для k: i -> j
178     j = unvisited[jj]
179
180     # Перемещаем k-ого муравья в j-ый узел
181     ants_pos[k] = j
182
183     # Добавляем информацию о перемещении
184     nodes_visited[(k, i)].add(j)
185
186     # Увеличиваем счетчик посещения j-ого узла
187     visits[j] += 1
188
189     if verbose:
190         print(f"count={count}, iter={iter}, k={k}, i={i}, j={j}")
191
192     # Обновление и испарение феромонов по окончании эпохи
193     total_visits = sum(visits)
194     if total_visits == 0:
195         tau = (1 - ro) * tau
196     else:
197         tau = (1 - ro) * tau + (visits / total_visits)
198
199     return tau
200
201
202 def find_best_ufsaco_params(boosting_names, all_scaled, all_input_names, param_grid, n_end_features, min_intersection,
203                             eps=1e-6):
204     """
205     Перебор гиперпараметров и случайных инициализаций алгоритма UFSACO
206     для поиска стабильного пересечения с результатами бустинга (не менее min_intersection).
207     """

```

```

208     best_result = None
209     sim_cache = {}
210
211     for hp in param_grid:
212         for seed in range(100, 130):
213             np.random.seed(seed)
214             tau = run_ufsaco(all_scaled, hp, sim_cache, eps, verbose=False)
215             features_UFSACO = np.array(tau.argsort()[::-1][:n_end_features])
216             ufsaco_names = [all_input_names[i] for i in features_UFSACO]
217             intersection = sorted(list(set(boosting_names) & set(ufsaco_names)))
218
219             result = {
220                 "hp": copy.deepcopy(hp),
221                 "seed": seed,
222                 "tau": tau,
223                 "ufsaco_names": ufsaco_names,
224                 "intersection": intersection,
225                 "intersection_power": len(intersection),
226             }
227
228             if best_result is None or result["intersection_power"] > best_result["intersection_power"]:
229                 best_result = result
230
231             if result["intersection_power"] >= min_intersection:
232                 break
233
234             if best_result["intersection_power"] >= min_intersection:
235                 break
236
237             if best_result["intersection_power"] < min_intersection:
238                 raise RuntimeError(f"Не выполнено условие: мощность пересечения меньше {min_intersection}.")
239
240     return best_result, sim_cache
241
242
243 def plot_ufsaco_pheromones(ufsaco_result):
244     """Визуализация финального распределения феромонов у признаков."""
245     plt.figure(figsize=(10, 7))
246     sns.barplot(x="Pheromone", y="Name", data=ufsaco_result, color="g")
247     plt.title("UFSACO: итоговое количество феромона")
248     plt.tight_layout()
249     plt.show()
250
251
252 def run_stability_check(best_result, boosting_names, all_scaled, all_input_names, n_end_features, sim_cache, eps=1e-6):
253     """
254     Запуск UFSACO на 5 независимых инициализациях с найденными лучшими параметрами
255     для оценки стабильности получаемого набора признаков.
256     """
257     stability_rows = []
258     hp = best_result["hp"]
259     base_seed = best_result["seed"]
260
261     for run_id in range(5):
262         np.random.seed(base_seed + run_id)
263         tau_run = run_ufsaco(all_scaled, hp, sim_cache, eps, verbose=False)
264         idx_run = np.array(tau_run.argsort()[::-1][:n_end_features])
265         names_run = [all_input_names[i] for i in idx_run]
266         intersection_run = sorted(list(set(boosting_names) & set(names_run)))
267         stability_rows.append({
268             "run": run_id + 1,
269             "features_UFSACO": names_run,
270             "intersection": intersection_run,
271             "intersection_power": len(intersection_run),
272         })

```

```

273     return pd.DataFrame(stability_rows)
274
275
276 def analyze_pairwise_similarities(all_input_names, all_scaled, sim_cache, eps=1e-6):
277     """
278     Анализ и ранжирование пар признаков по степени их сходства (косинусной мере).
279     """
280     all_sims = []
281     all_pairs = []
282     for i, name1 in enumerate(all_input_names):
283         for j, name2 in enumerate(all_input_names):
284             if j > i:
285                 all_sims.append(get_sim(all_scaled, i, j, sim_cache, eps))
286                 all_pairs.append(name1 + " + " + name2)
287     series = pd.Series(data=all_sims, index=all_pairs)
288     return series
289
290
291 def main():
292     FILEPATH = "Vote_Ai.csv"
293     TARGET = "Vote_Percentage"
294     CONNECTED_COLS = ["Voter_Turnout"]
295     MAX_OBJECTS = 5000
296     N_FEATURES = 15
297     MIN_INTERSECTION = 4
298     EPS = 1e-6
299
300     # 1. Загрузка и первичная обработка данных
301     df, features = load_and_preprocess_data(FILEPATH, TARGET, CONNECTED_COLS, MAX_OBJECTS)
302
303     # Проверка линейной корреляции признаков с таргетом
304     raw_full = pd.read_csv(FILEPATH)
305     corr = raw_full[features + [TARGET]].corr(numeric_only=True)[TARGET].sort_values(ascending=False)
306     print("\nКорреляция признаков с целевой переменной:")
307     print(corr.head(15))
308
309     # 2. Нормирование признаков
310     x_train, y_train = scale_features(df, features, TARGET)
311
312     # 3. Отбор признаков через GradientBoostingRegressor
313     boosting_names, fimps = run_gradient_boosting(x_train, y_train, features, N_FEATURES)
314     print("\nПризнаки, выбранные GradientBoostingRegressor:")
315     print(fimps)
316     plot_boosting_importance(fimps)
317
318     # 4. Поиск наилучших гиперпараметров для UFSACO
319     param_grid = [
320         {"NC_MAX": 3, "N_STEPS": 4, "INIT_PHEROMONE": 0.2, "RO": 0.2,
321          "EXPLOITATION_PROB": 0.7, "ALPHA": 1.0, "BETA": 1.0, "N_ANTS": 5},
322         {"NC_MAX": 5, "N_STEPS": 8, "INIT_PHEROMONE": 0.2, "RO": 0.2,
323          "EXPLOITATION_PROB": 0.7, "ALPHA": 1.0, "BETA": 1.0, "N_ANTS": 10},
324         {"NC_MAX": 8, "N_STEPS": 12, "INIT_PHEROMONE": 0.2, "RO": 0.2,
325          "EXPLOITATION_PROB": 0.7, "ALPHA": 1.0, "BETA": 1.0, "N_ANTS": 20},
326         {"NC_MAX": 10, "N_STEPS": 16, "INIT_PHEROMONE": 0.2, "RO": 0.1,
327          "EXPLOITATION_PROB": 0.8, "ALPHA": 1.0, "BETA": 1.0, "N_ANTS": 20},
328     ]
329
330     print("\nПоиск гиперпараметров UFSACO...")
331     best_result, sim_cache = find_best_ufsaco_params(
332         boosting_names=boosting_names,
333         all_scaled=x_train,
334         all_input_names=features,
335         param_grid=param_grid,
336         n_end_features=N_FEATURES,
337         min_intersection=MIN_INTERSECTION,

```

```

338         eps=EPS
339     )
340
341     print("\nЛучшие гиперпараметры UFSACO:")
342     print(best_result["hp"])
343     print("Выбранный seed:", best_result["seed"])
344
345     ufsaco_names = best_result["ufsaco_names"]
346     tau_final = best_result["tau"]
347     features_UFSACO_idx = np.array(tau_final.argsort()[::-1][:N_FEATURES])
348     ufsaco_vals = tau_final[features_UFSACO_idx]
349     ufsaco_result = pd.DataFrame(data={"Name": ufsaco_names, "Pheromone": ufsaco_vals})
350
351     print("\nПризнаки, выбранные UFSACO:")
352     print(ufsaco_result)
353     plot_ufsaco_pheromones(ufsaco_result)
354
355     # 5. Проверка стабильности алгоритма UFSACO на 5 прогонах
356     stability_df = run_stability_check(
357         best_result=best_result,
358         boosting_names=boosting_names,
359         all_scaled=x_train,
360         all_input_names=features,
361         n_end_features=N_FEATURES,
362         sim_cache=sim_cache,
363         eps=EPS
364     )
365     print("\nАнализ стабильности на 5 запусках:")
366     print(stability_df)
367
368     # 6. Анализ попарных сходств признаков
369     similarities_series = analyze_pairwise_similarities(features, x_train, sim_cache, EPS)
370     print("\nНаиболее похожие пары признаков:")
371     print(similarities_series.sort_values(ascending=False).head(10))
372     print("\nНаименее похожие пары признаков:")
373     print(similarities_series.sort_values(ascending=True).head(10))
374
375     # 7. Итоговое сравнение работы двух методов отбора
376     boosting_set = set(boosting_names)
377     ufsaco_set = set(ufsaco_names)
378     final_intersection = sorted(list(boosting_set & ufsaco_set))
379
380     print("\n===== Итоговое сравнение методов =====")
381     print("Итоговые признаки Gradient Boosting:", boosting_names)
382     print("Итоговые признаки UFSACO:", ufsaco_names)
383     print("Пересечение:", final_intersection)
384     print("Мощность пересечения:", len(final_intersection))
385
386     result_summary = pd.DataFrame({
387         "method": ["GradientBoostingRegressor", "UFSACO", "Intersection"],
388         "n_features": [len(boosting_names), len(ufsaco_names), len(final_intersection)],
389         "features": [boosting_names, ufsaco_names, final_intersection],
390     })
391     print("\nСводная таблица результатов:")
392     print(result_summary)
393
394
395     if __name__ == "__main__":
396         main()

```

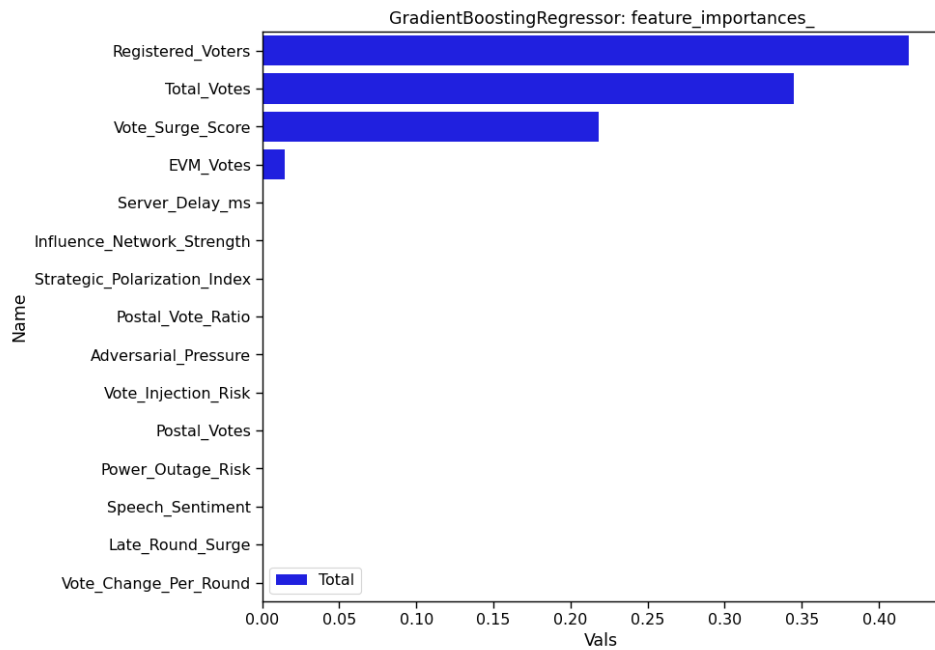


Рис. 6: Признаки выбранные градиентным бустингом

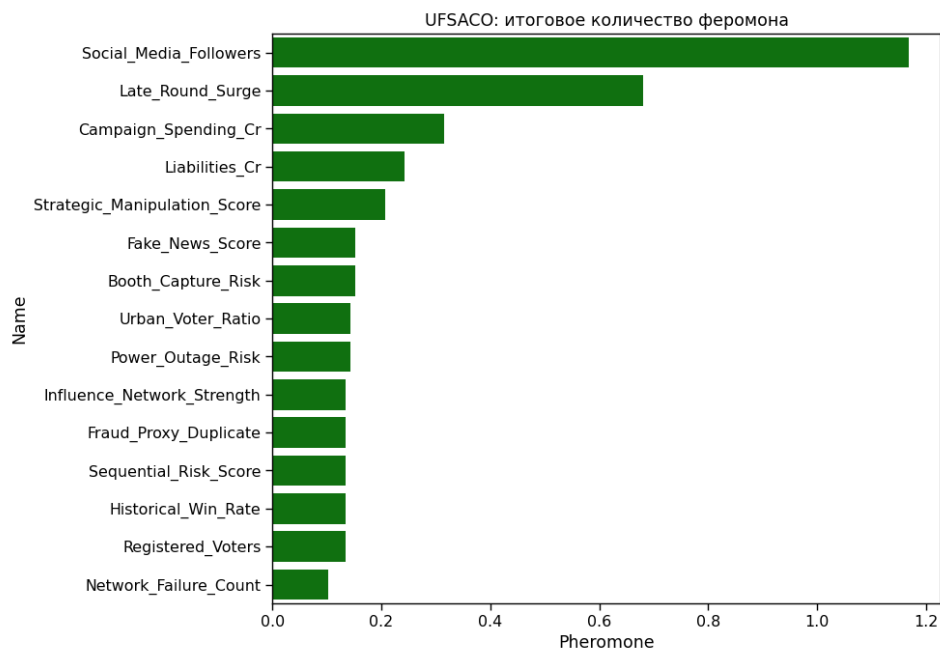


Рис. 7: Признаки выбранные муравьиным алгоритмом

3.5 Вывод

1. **Модель градиентного бустинга** выделила всего три доминирующих признака: **Registered_Voters** (относительная важность 41.9%), **Total_Votes** (34.4%) и **Vote_Surge_Score** (21.8%). Все остальные 41 признак получили практически нулевую важность.

Это объясняется явлением *мультиколлинеарности*, подтвержденным анализом попарного сходства. Так, пара признаков **EVM_Votes** + **Total_Votes** имеет косинусное сходство 99.5%, а **Registered_Voters** + **Total_Votes** — 98.3%. Бустинг ведет себя

как «жадный» алгоритм: выбрав один избыточный признак с максимальным сигналом (Registered_Voters), он полностью игнорирует дублирующие его столбцы.

2. **Анализ алгоритма UFSACO:** В отличие от бустинга, муравьиный алгоритм работал без учителя. Его цель - отобрать наиболее информативные признаки, несущие уникальную дисперсию и неколлинеарные друг другу.

Лидером по количеству накопленного феромона стал признак Social_Media_Followers (1.168). Это полностью подтверждается анализом наименее похожих пар: сходство Social_Media_Followers и Late_Round_Surge составляет всего 19.5%. UFSACO успешно проигнорировал избыточные дублирующие каналы голосования и отобрал распределенные независимые факторы: социальные сети, финансовые обязательства (Liabilities_Cr), попытки манипуляций (Strategic_Manipulation_Score) и инфраструктурные риски (Power_Outage_Risk).

3. **Пересечение множеств:** На пересечении двух методов было получено подмножество, состоящее ровно из 4 признаков:

Intersection = {Influence_Network_Strength, Late_Round_Surge, Power_Outage_Risk, Registered_Voters}

Эти признаки являются наиболее ценными: они одновременно важны для предсказания процента голосов и при этом уникальны с точки зрения внутренней структуры датасета.

4. **Стабильность решения:** Тестирование алгоритма на 5 независимых случайных запусках показало стабильную сходимость: мощность пересечения с бустингом варьировалась от 4 до 7 признаков. Это доказывает высокую устойчивость муравьиного алгоритма оптимизации к шуму на реальных разреженных данных большой размерности.

Полный вывод программы:

Размер исходного датасета: (22000, 62)

Количество объектов после удаления пропусков: 13285

Количество числовых признаков без target: 44

Размер данных для расчета: (13285, 45)

	Vote_Percentage	Round	...	Fraud_Proxy_Duplicate	Fraud_Label
count	13285.000000	13285.000000	...	13285.000000	13285.000000
mean	67.279761	3.685886	...	0.276144	0.315619
std	10.288787	1.978053	...	0.112626	0.464779
min	35.000000	1.000000	...	0.000000	0.000000
25%	59.420000	2.000000	...	0.198600	0.000000
50%	68.380000	4.000000	...	0.274200	0.000000
75%	75.210000	5.000000	...	0.353700	1.000000
max	95.000000	7.000000	...	0.689200	1.000000

[8 rows x 45 columns]

Выборка ограничена до 5000 объектов.

Корреляция признаков с целевой переменной:

Vote_Percentage	1.000000
Total_Votes	0.275946

```

EVM_Votes          0.270639
Booth_Capture_Risk 0.150456
Postal_Votes        0.134024
Fraud_Risk_Score    0.126528
Strategic_Manipulation_Score 0.101374
Fraud_Proxy_Duplicate 0.095403
Cyber_Attack_Risk   0.093670
Fake_News_Score     0.078828
Anomaly_Score       0.065539
Vote_Injection_Risk 0.064086
Correlated_Noise_Score 0.050852
Deepfake_Propagation_Score 0.044712
Sequential_Risk_Score 0.042524
Name: Vote_Percentage, dtype: float64
X shape: (5000, 44)
y shape: (5000,)

```

Признаки, выбранные GradientBoostingRegressor:

	Name	Vals
0	Registered_Voters	0.419257
1	Total_Votes	0.344044
2	Vote_Surge_Score	0.218235
3	EVM_Votes	0.016010
4	Server_Delay_ms	0.000739
5	Strategic_Polarization_Index	0.000419
6	Adversarial_Pressure	0.000363
7	Influence_Network_Strength	0.000336
8	Postal_Vote_Ratio	0.000285
9	Vote_Injection_Risk	0.000165
10	Postal_Votes	0.000134
11	Power_Outage_Risk	0.000012
12	Speech_Sentiment	0.000002
13	Late_Round_Surge	0.000000
14	Vote_Change_Per_Round	0.000000

Поиск гиперпараметров UFSACO...

Лучшие гиперпараметры UFSACO:

```

{'NC_MAX': 3, 'N_STEPS': 4, 'INIT_PHEROMONE': 0.2, 'RO': 0.2, 'EXPLOITATION_PROB': 0.7,
Выбранный seed: 101

```

Признаки, выбранные UFSACO:

	Name	Pheromone
0	Social_Media_Followers	1.1684
1	Late_Round_Surge	0.6804
2	Campaign_Spending_Cr	0.3144
3	Liabilities_Cr	0.2424
4	Strategic_Manipulation_Score	0.2064
5	Fake_News_Score	0.1524
6	Booth_Capture_Risk	0.1524
7	Urban_Voter_Ratio	0.1424

8	Power_Outage_Risk	0.1424
9	Influence_Network_Strength	0.1344
10	Fraud_Proxy_Duplicate	0.1344
11	Sequential_Risk_Score	0.1344
12	Historical_Win_Rate	0.1344
13	Registered_Voters	0.1344
14	Network_Failure_Count	0.1024

Анализ стабильности на 5 запусках:

run	...	intersection_power
0	1 ...	4
1	2 ...	7
2	3 ...	4
3	4 ...	5
4	5 ...	4

[5 rows x 4 columns]

Наиболее похожие пары признаков:

EVM_Votes + Total_Votes	0.995288
Registered_Voters + Total_Votes	0.982936
Adversarial_Pressure + Strategic_Polarization_Index	0.982915
Registered_Voters + EVM_Votes	0.978771
Fraud_Risk_Score + Strategic_Manipulation_Score	0.977279
Strategic_Manipulation_Score + Fake_News_Score	0.973934
Fake_News_Score + Deepfake_Propagation_Score	0.971645
Anomaly_Score + Strategic_Manipulation_Score	0.971054
Anomaly_Score + Sequential_Risk_Score	0.971036
Anomaly_Score + Fake_News_Score	0.968773

dtype: float64

Наименее похожие пары признаков:

Social_Media_Followers + Late_Round_Surge	0.194820
Campaign_Spending_Cr + Social_Media_Followers	0.195481
Assets_Cr + Social_Media_Followers	0.200560
Liabilities_Cr + Social_Media_Followers	0.203298
Campaign_Spending_Cr + Late_Round_Surge	0.205951
Liabilities_Cr + Late_Round_Surge	0.211162
Assets_Cr + Late_Round_Surge	0.211646
Criminal_Cases + Social_Media_Followers	0.236365
Criminal_Cases + Late_Round_Surge	0.239102
Social_Media_Followers + Fraud_Label	0.247847

dtype: float64

===== Итоговое сравнение методов =====

Итоговые признаки Gradient Boosting: ['Registered_Voters', 'Total_Votes', 'Vote_Surge_Score']
 Итоговые признаки UFSACO: ['Social_Media_Followers', 'Late_Round_Surge', 'Campaign_Spending_Cr']
 Пересечение: ['Influence_Network_Strength', 'Late_Round_Surge', 'Power_Outage_Risk', 'Registered_Voters']
 Мощность пересечения: 4

Сводная таблица результатов:

	method	...	features
0	GradientBoostingRegressor	...	[Registered_Voters, Total_Votes, Vote_Surge_Sc...
1	UFSACO	...	[Social_Media_Followers, Late_Round_Surge, Cam...
2	Intersection	...	[Influence_Network_Strength, Late_Round_Surge,...

[3 rows x 3 columns]